Task Report: Tooth Number Detection using YOLO11m

Candidate Name: Satyam Yadav

Contact: +91 9026008686

Email: satyamy9794@gmail.com

Demo Application: Yolo Tooth Numbering app

GitHub: sattyaaa/tooth-numbering-yolo

1. Objective

The goal of this project was to train an object detection model that can identify and number teeth from dental X-ray images. Since the task involves detecting many small, closely placed objects, I chose a YOLO-based approach because it is fast, accurate, and well suited for real-time detection.

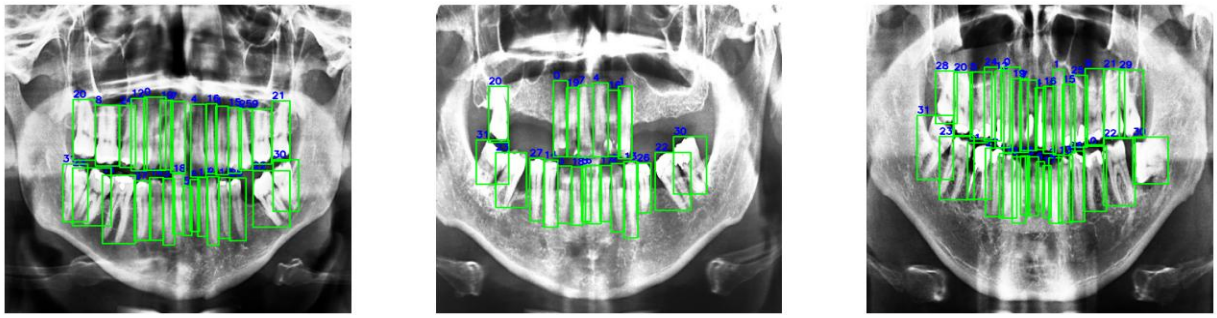
2. Dataset Overview

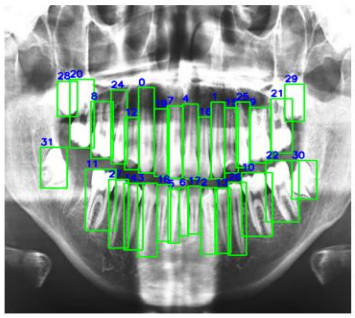
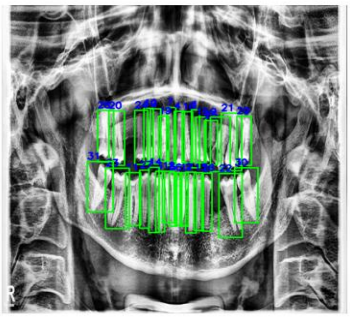
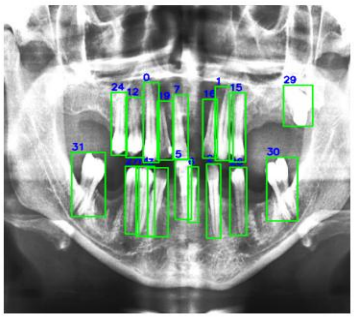
The provided dataset contained about 500 dental images with bounding-box annotations in YOLO format. The annotations were split into **32 tooth classes**.

After extraction, I verified the folder structure, collected all image files, and created an **80/20 train-validation split**. This gave: - **400 images for training** - **100 images for validation**

The validation set contained **2,747 annotated tooth instances**, which provided a solid basis for evaluation.

Sample Training Images with Ground Truth Boxes



3. Data Preparation

Before training, I prepared the dataset in a format that YOLO could use directly.

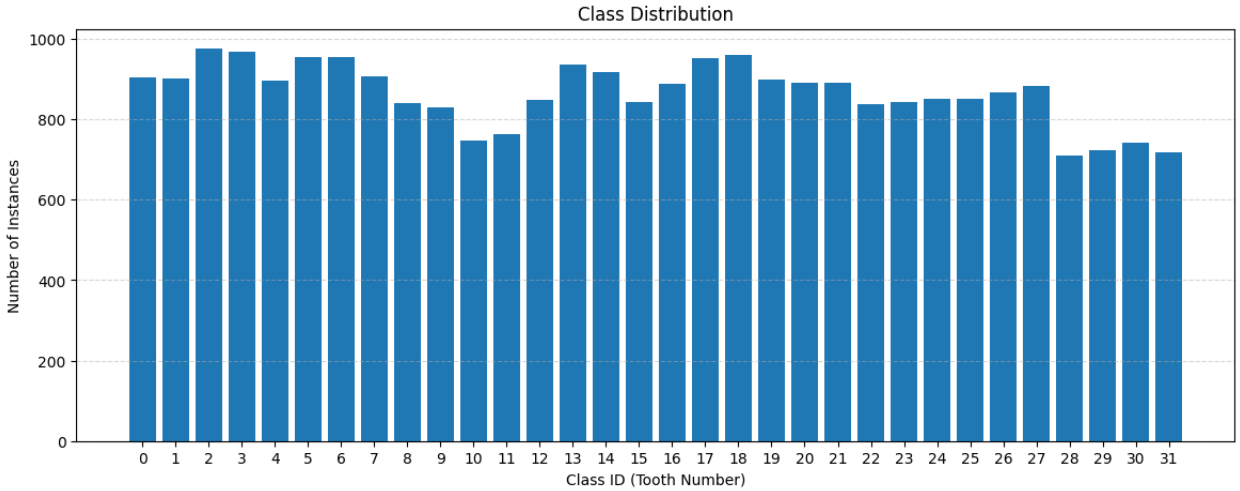
The steps were: - Unzipped the dataset - Checked that image files and label files matched correctly - Split the data into train and validation folders - Built the `data.yaml` configuration file - Confirmed that GPU support was available before starting training

I also visualized random training images with ground-truth boxes to make sure the annotations were aligned properly with the teeth in the X-ray images.

4. Dataset Analysis

I checked the class distribution across all label files to understand whether the dataset was balanced.

The class count was fairly even across most tooth numbers, which helped training remain stable. A few classes had slightly fewer examples than others, but overall the dataset did not show severe imbalance. This was useful because the main challenge was not class skew, but rather the visual difficulty of dental X-rays themselves, such as overlap, blur, and low contrast.



5. Model Choice

For training, I selected **YOLO11m**.

I chose this model because: - it is stronger than smaller variants like YOLO11s, - it offers a good balance between accuracy and speed, - it works well for small-object detection, - and the pretrained weights make transfer learning effective on a relatively small dataset.

For this task, I felt YOLO11m was a practical choice because tooth detection needs both fine localization and reliable class prediction.

6. Training Setup

The model was trained using the Ultralytics YOLO training pipeline with the following settings:

- **Model:** `yolo11m.pt`
- **Image size:** 768
- **Batch size:** 12
- **Epochs:** 100
- **Patience:** 20
- **Cosine learning rate:** enabled

The model was trained on the custom dataset using the `data.yaml` file that mapped all 32 tooth classes.

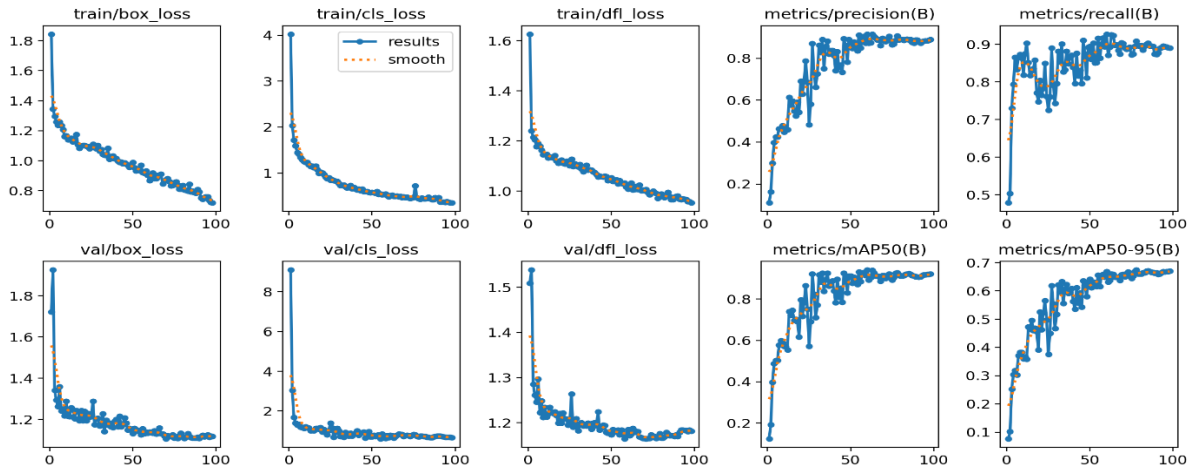
During training, I monitored the loss curves and metric plots to check whether the model was converging properly. The curves showed steady improvement over time, which indicated that the model was learning the tooth patterns rather than memorizing the data.

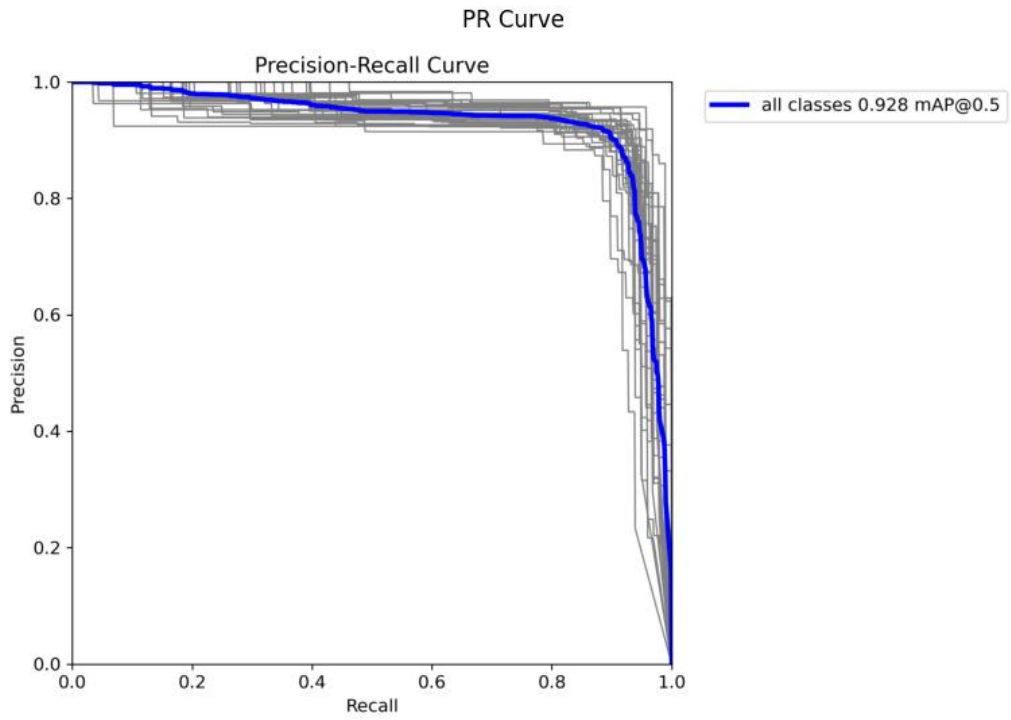
7. Validation Results

After training, I evaluated the best model on the validation set.

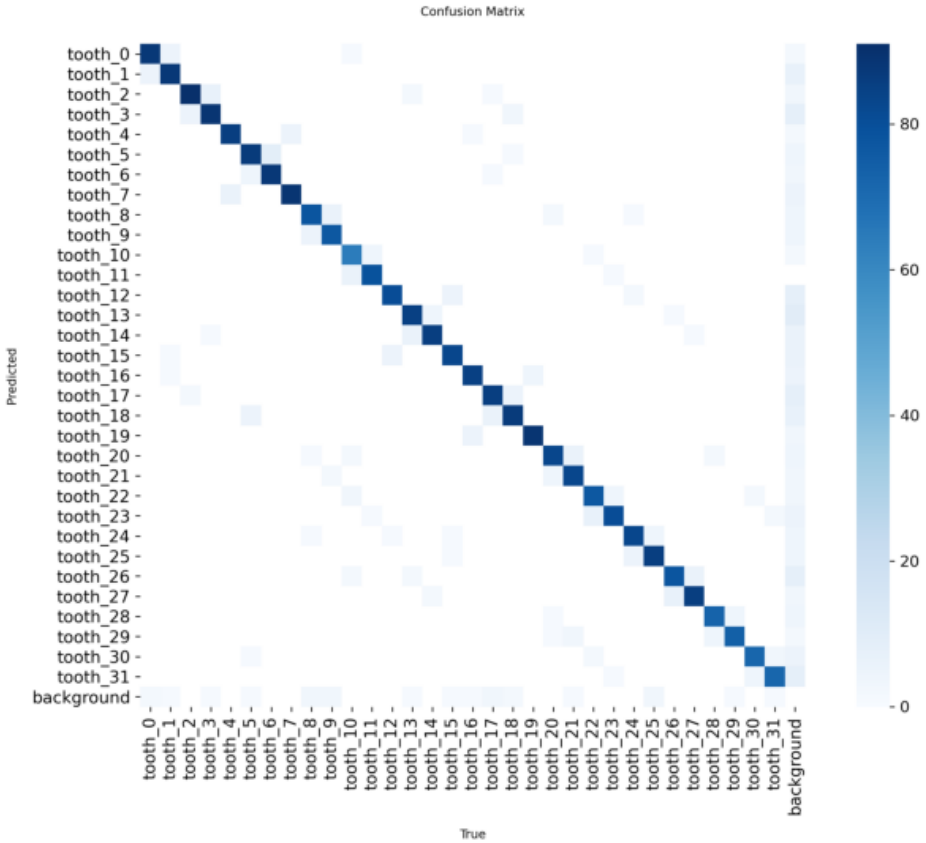
The main results were: - **Precision:** 0.981 - **Recall:** 0.981 - **mAP@0.5:** 0.986 - **mAP@0.5:0.95:** 0.816

These numbers show that the model performed strongly on the validation images. The high precision and recall suggest that the model was able to detect most teeth correctly with few false detections. The mAP@0.5:0.95 score is lower than mAP@0.5, which is expected because stricter IoU thresholds make localization harder, especially in crowded dental X-rays.


train/box_loss
train/cls_loss
results
smooth
train/dfl_loss
metrics/precision(B)
metrics/recall(B)
val/box_loss
val/cls_loss
val/dfl_loss
metrics/mAP50(B)
metrics/mAP50-95(B)


PR Curve
Precision-Recall Curve
all classes 0.928 mAP@0.5
Precision
Recall

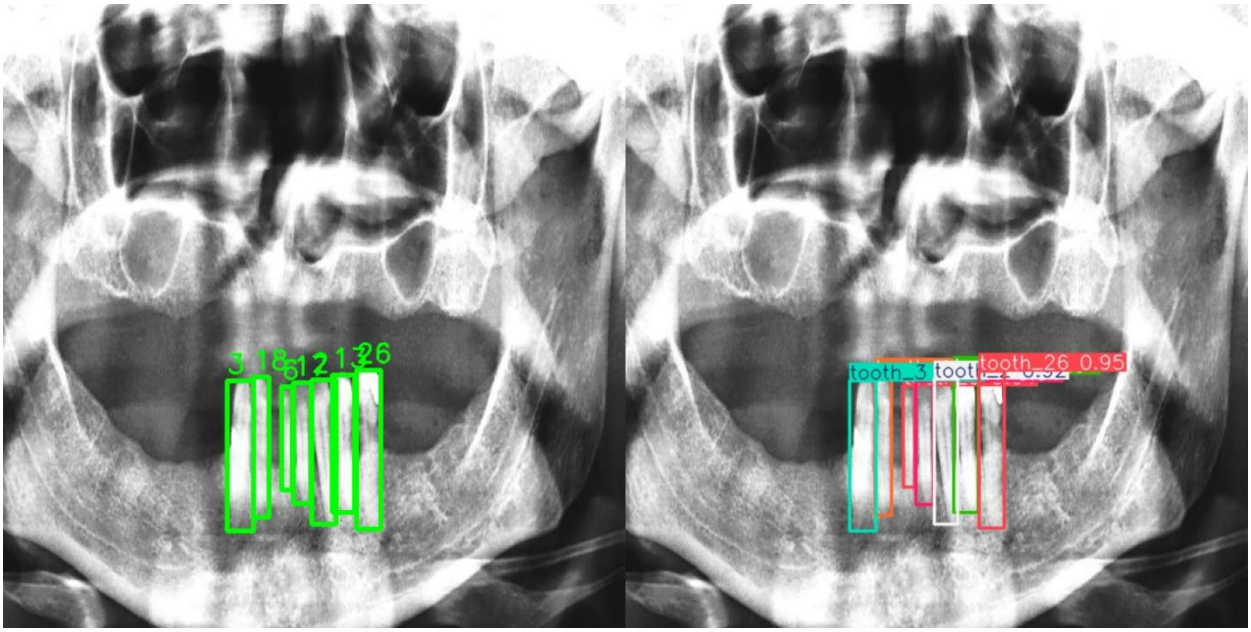
Confusion Matrix

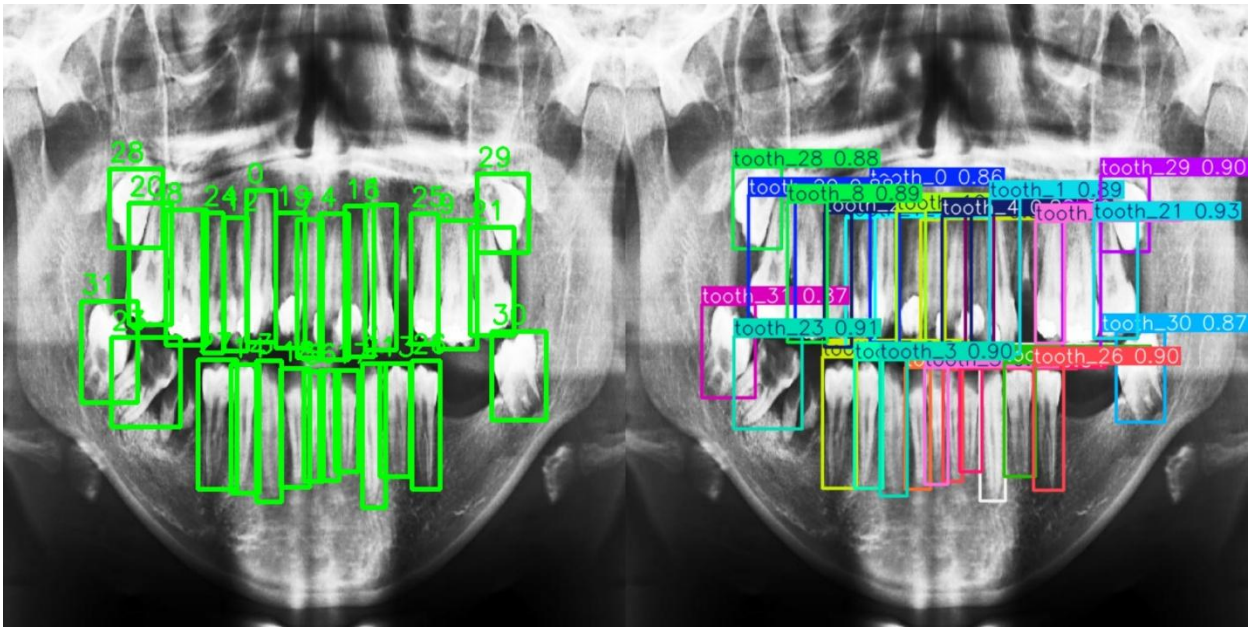


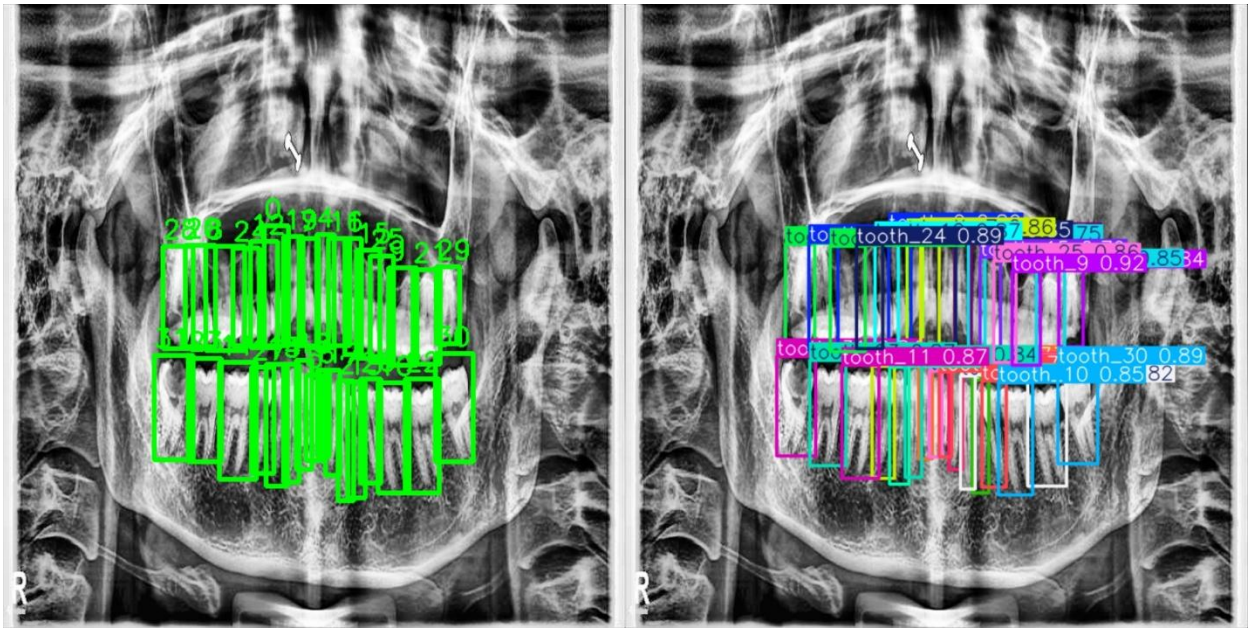
8. Qualitative Prediction Review

To check the model visually, I ran inference on random validation images and compared the predicted boxes with the ground-truth labels.

From these sample outputs, the model generally detected tooth positions well and assigned the correct class numbers in most cases. The predictions looked especially strong in clearer images where teeth were well separated.


tooth_26 0.95


tooth_28 0.88
tooth_29 0.90
tooth_21 0.93
tooth_23 0.91
tooth_26 0.90



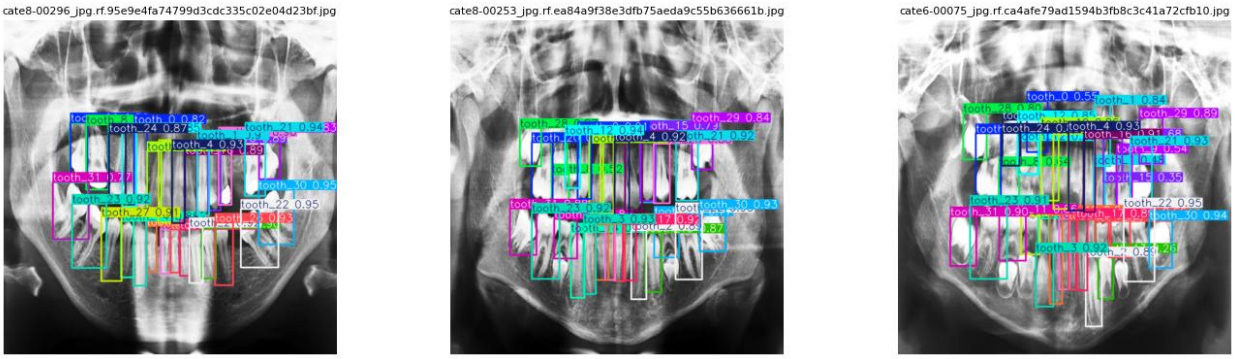
9. Error Analysis

I also looked at failure cases to understand where the model struggled.

The main difficult cases were: - overlapping teeth, - blurred X-rays, - low-contrast images, - and images where tooth boundaries were not very clear.

These cases are common in medical imaging and explain most of the remaining errors. In some difficult images, the model missed a few teeth or made slightly inaccurate box placements.

Potential Detection Failure Cases



10. Inference Speed

I tested inference speed on 20 random validation images.

The average inference time was: - **0.0408 seconds per image** - approximately **24.53 FPS**

This is a good result for practical use, since it shows the model can run efficiently and could be useful in a clinic-style workflow where quick feedback is important.

11. Conclusion

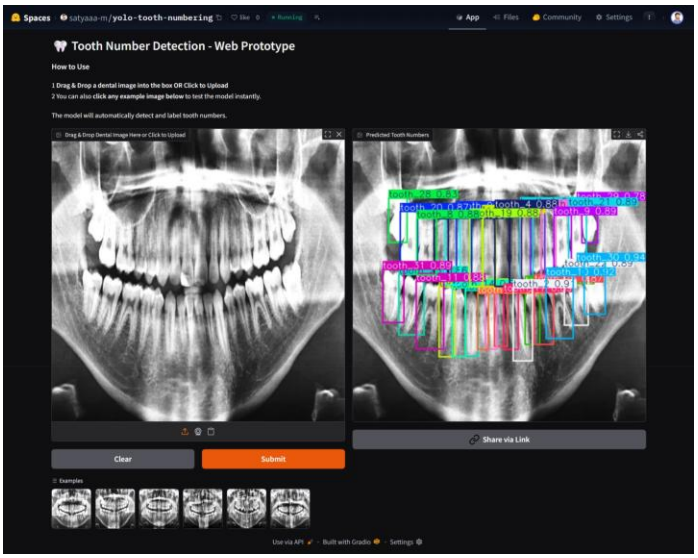
This project showed that YOLO11m can detect and number teeth from dental X-ray images with strong accuracy and good runtime performance. The model achieved high precision and recall on the validation set, and the qualitative results confirmed that it learns meaningful dental features.

Overall, this was a useful object detection task because it combined computer vision, model training, dataset handling, and practical evaluation in a medical imaging setting.

12. Future Improvements

A few improvements could make the model even better: - train with higher resolution images, - try segmentation-based detection for tighter localization, - experiment with k-fold validation, - compare YOLO11m with other YOLO variants, - and use more focused augmentation for dental X-rays.

13. Submission Contents

The final submission includes:

- source code for training and inference
- trained model weights
- sample prediction outputs
- a working web demo
- and this report.